

Docling 文档解析工具调研方案

目录

- **Docling 文档解析工具调研方案**
 - 一、概述
 - 1.1 工具简介
 - 1.2 适用场景
 - 二、安装部署
 - 2.1 环境要求
 - 2.2 基础安装
 - 2.3 可选功能安装
 - 2.4 OCR 系统依赖（可选，处理扫描件需要）
 - 三、支持的文档格式
 - 四、合同文档解析方案
 - 4.1 整体架构
 - 4.2 基础解析代码
 - 4.3 按章节解析合同
 - 4.4 导出为 Markdown / HTML / TXT
 - 4.5 可视化到 HTML 页面（带导航目录）
 - 五、合同文档章节解析 Demo
 - 5.1 完整示例代码
 - 5.2 运行方式
 - 5.3 输出结果说明
 - 六、HybridChunker 按章节分块（适用于 RAG）

Docling 文档解析工具调研方案

一、概述

1.1 工具简介

Docling 是 IBM Research Zurich 开发的开源文档解析工具，现托管于 LF AI & Data Foundation。它利用计算机视觉模型理解文档结构，而非简单提取原始文本，能够完整保留文档的层级结构、阅读顺序、表格、图片等语义信息。

核心优势：

- 本地运行，支持离线/隔离环境，无需将数据发送到第三方服务
- 统一的 DoclingDocument 数据模型 (Pydantic)，保留文档层级结构与元数据
- 底层使用两个 AI 模型：Layout Analysis（识别标题/正文/表格/图片布局）和 TableFormer（识别表格行列结构）
- 与 LangChain、LlamaIndex、Haystack 等 AI 框架原生集成

1.2 适用场景

- 合同文档 (PDF/Word) 的结构化解析与章节提取
- 文档内容可视化展示 (HTML 页面渲染)
- RAG 知识库构建中的文档预处理
- 表格、图片的自动提取与分析

二、安装部署

2.1 环境要求

- Python ≥ 3.10
- 推荐 pip ≥ 23.0

2.2 基础安装

```
</> Bash |
1 # 标准安装
2 pip install docling
3
4 # Linux CPU-only (无 GPU 推荐)
5 pip install docling --extra-index-url https://download.pytorch.org/whl/cpu
```

2.3 可选功能安装

```
</> Bash |
1 pip install "docling[vlm]"          # 视觉语言模型支持 (图片理解)
2 pip install "docling[easyocr]"      # EasyOCR 引擎 (扫描件 OCR)
3 pip install "docling[tesseract]"    # Tesseract OCR 引擎
```

2.4 OCR 系统依赖 (可选, 处理扫描件需要)

```
</> Bash |
1 # Ubuntu/Debian
2 apt-get install tesseract-ocr tesseract-ocr-eng libtesseract-dev liblibleptonica-dev pkg-config
```

```
3 export TESSDATA_PREFIX=$(dpkg -L tesseract-ocr-eng | grep tessdata$)
4
5 # macOS
6 brew install tesseract leptonica pkg-config
7 export TESSDATA_PREFIX=/opt/homebrew/share/tessdata/
```

注意： 首次运行会自动下载预训练模型（约 500MB），之后缓存本地，无需重复下载。

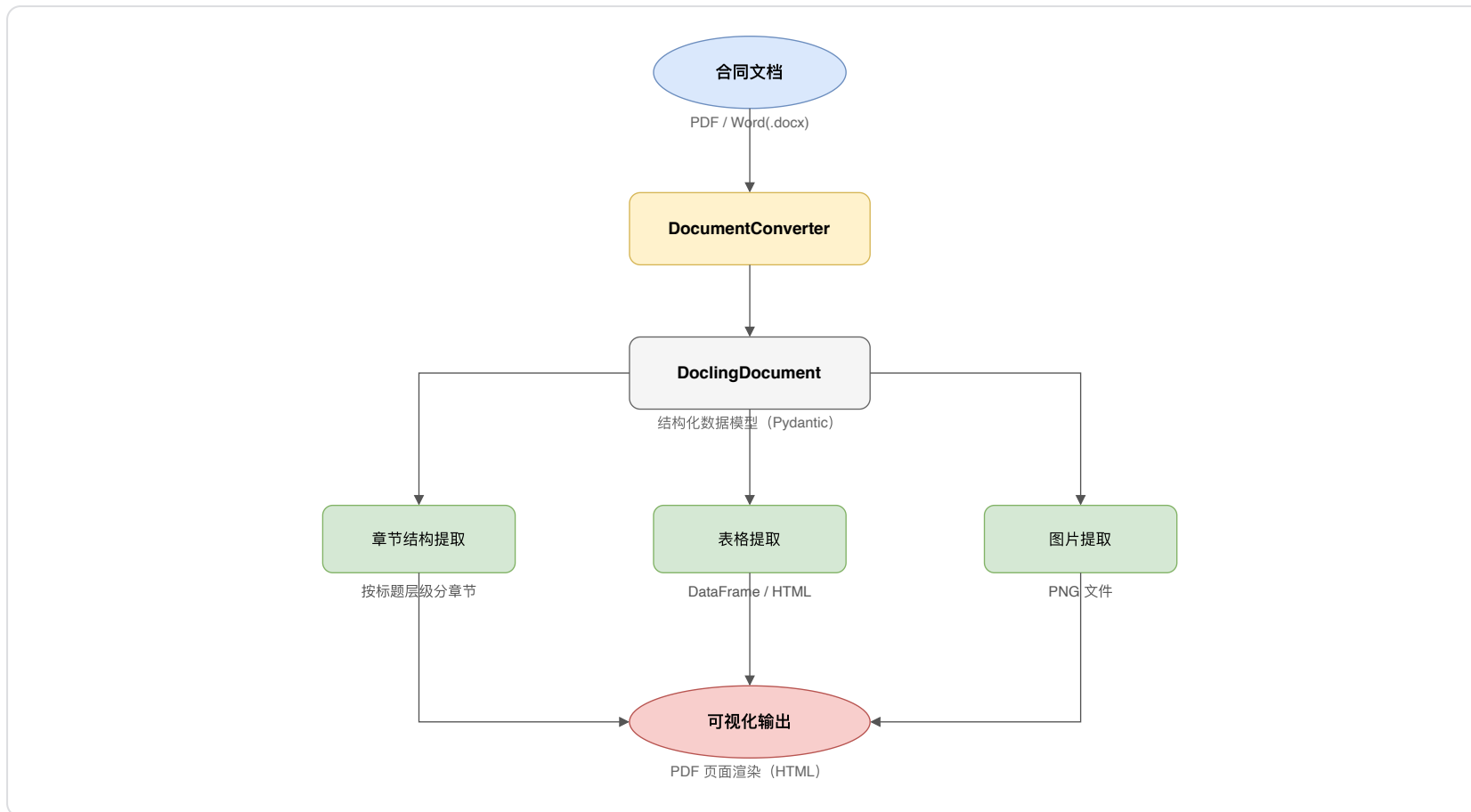
三、支持的文档格式

1	输入类型	支持格式
2	Office 文档	DOCX、XLSX、PPTX
3	PDF	PDF（含扫描件，需开启 OCR）
4	网页/标记语言	HTML、Markdown、AsciiDoc、LaTeX
5	电子书	EPUB
6	图片	PNG、JPEG、TIFF、BMP、WEBP
7	纯文本	.txt、.text
8	专业 XML	JATS 论文、XBRL 财务报告

输出格式： Markdown、HTML、JSON（无损序列化）、纯文本

四、合同文档解析方案

4.1 整体架构



4.2 基础解析代码

</>

Python

```
1 from docling.document_converter import DocumentConverter, PdfFormatOption
2 from docling.datamodel.base_models import InputFormat
3 from docling.datamodel.pipeline_options import PdfPipelineOptions
4
```

```

5 # 配置 PDF 解析选项
6 pipeline_options = PdfPipelineOptions()
7 pipeline_options.do_ocr = False          # 数字 PDF 无需 OCR
8 pipeline_options.do_table_structure = True # 开启表格结构识别
9 pipeline_options.generate_picture_images = True # 提取图片
10
11 converter = DocumentConverter(
12     format_options={
13         InputFormat.PDF: PdfFormatOption(pipeline_options=pipeline_options)
14     }
15 )
16
17 # 解析合同文档
18 result = converter.convert("contract.pdf") # 或 contract.docx
19 doc = result.document

```

4.3 按章节解析合同

Docling 的 DoclingDocument 内置层级树结构，`label` 字段标识每个元素的类型：

- `SECTION_HEADING` / `TITLE`：章节标题
- `TEXT`：正文段落
- `TABLE`：表格
- `PICTURE`：图片

</>

Python

```

1 def parse_contract_by_section(doc):
2     """按章节解析合同文档，返回章节树"""
3     sections = []

```

```
4     current_section = None
5
6     for item in doc.texts:
7         label = str(item.label)
8         text = item.text.strip()
9         if not text:
10             continue
11
12         if label in ("section_heading", "title"):
13             current_section = {
14                 "heading": text,
15                 "level": getattr(item, 'level', 1),
16                 "content": []
17             }
18             sections.append(current_section)
19         elif current_section is not None:
20             current_section["content"].append(text)
21         else:
22             sections.append({"heading": "前言", "level": 0, "content": [text]})
23
24     return sections
25
26 sections = parse_contract_by_section(doc)
27 for sec in sections:
28     print(f"{'#' * max(sec['level'], 1)} {sec['heading']}")
29     for para in sec['content']:
30         print(para)
31     print()
```

4.4 导出为 Markdown / HTML / TXT

</>

Python

```
1 # 导出为 Markdown (保留标题层级)
2 md_content = doc.export_to_markdown()
3 with open("contract.md", "w", encoding="utf-8") as f:
4     f.write(md_content)
5
6 # 导出为 HTML (可直接在浏览器可视化)
7 html_content = doc.export_to_html()
8 with open("contract.html", "w", encoding="utf-8") as f:
9     f.write(html_content)
10
11 # 导出为纯文本
12 txt_content = doc.export_to_text()
13 with open("contract.txt", "w", encoding="utf-8") as f:
14     f.write(txt_content)
```

4.5 可视化到 HTML 页面 (带导航目录)

</>

Python

```
1 def export_contract_with_toc(doc, output_path: str):
2     """导出带章节目录的合同 HTML 页面"""
3     sections = parse_contract_by_section(doc)
4
5     toc_items = ""
6     for i, sec in enumerate(sections):
```



```
7         indent = "padding-left:" + str((sec['level'] - 1) * 20) + "px"
8         toc_items += f'<li style="{indent}"><a href="#sec-{i}">{sec["heading"]}</a></li>\n'
9
10    body_items = ""
11    for i, sec in enumerate(sections):
12        h_tag = f"h{min(sec['level'] + 1, 4)}" if sec['level'] > 0 else "h2"
13        body_items += f'<{h_tag} id="sec-{i}">{sec["heading"]}</{h_tag}>\n'
14        for para in sec['content']:
15            body_items += f'<p>{para}</p>\n'
16
17    html = f'<!DOCTYPE html>
18    <html lang="zh-CN">
19    <head>
20    <meta charset="UTF-8">
21    <title>合同文档</title>
22    <style>
23    body {{ font-family: 'Microsoft YaHei', sans-serif; margin: 0; display: flex; }}
24    #toc {{ width: 260px; min-height: 100vh; background: #f5f5f5; padding: 20px;
25        position: sticky; top: 0; overflow-y: auto; }}
26    #toc ul {{ list-style: none; padding: 0; }}
27    #toc a {{ text-decoration: none; color: #333; font-size: 14px; }}
28    #toc a:hover {{ color: #1677ff; }}
29    #content {{ flex: 1; padding: 40px; max-width: 900px; }}
30    h2 {{ color: #1a1a1a; border-bottom: 2px solid #1677ff; padding-bottom: 8px; }}
31    h3 {{ color: #333; }}
32    p {{ line-height: 1.8; color: #555; }}
33    </style>
34    </head>
35    <body>
```

```
36 <nav id="toc"><h3>目录</h3><ul>{toc_items}</ul></nav>
37 <main id="content">{body_items}</main>
38 </body>
39 </html>"""
40
41     with open(output_path, "w", encoding="utf-8") as f:
42         f.write(html)
43
44 # 使用示例
45 export_contract_with_toc(doc, "contract_visual.html")
```

五、合同文档章节解析 Demo

5.1 完整示例代码

</>

Python |

```
1 #!/usr/bin/env python3
2 """
3 合同文档章节解析 Demo
4 支持: PDF / Word(.docx) 输入
5 输出: 按章节解析的 TXT / HTML / Markdown
6 """
7 from pathlib import Path
8 from docling.document_converter import DocumentConverter, PdfFormatOption
9 from docling.datamodel.base_models import InputFormat
10 from docling.datamodel.pipeline_options import PdfPipelineOptions
11
```

```
12
13 def parse_contract(file_path: str, output_dir: str = "./output"):
14     output_path = Path(output_dir)
15     output_path.mkdir(exist_ok=True)
16
17     pipeline_options = PdfPipelineOptions()
18     pipeline_options.do_table_structure = True
19     converter = DocumentConverter(
20         format_options={InputFormat.PDF: PdfFormatOption(pipeline_options=pipeline_options)}
21     )
22
23     print(f"正在解析: {file_path}")
24     result = converter.convert(file_path)
25     doc = result.document
26
27     # ---- 1. 按章节解析 ----
28     sections = []
29     current = None
30     for item in doc.texts:
31         label = str(item.label)
32         text = item.text.strip()
33         if not text:
34             continue
35         if label in ("section_heading", "title"):
36             current = {"heading": text, "level": getattr(item, "level", 1), "content": []}
37             sections.append(current)
38         elif current:
39             current["content"].append(text)
40
```

```

41 # ---- 2. 导出 TXT (按章节) ----
42 txt_lines = []
43 for sec in sections:
44     txt_lines.append("=" * 60)
45     txt_lines.append(f" 【{sec['heading']}】 ")
46     txt_lines.append("=" * 60)
47     txt_lines.extend(sec["content"])
48     txt_lines.append("")
49 txt_out = output_path / "contract_chapters.txt"
50 txt_out.write_text("\n".join(txt_lines), encoding="utf-8")
51 print(f"TXT 已保存: {txt_out}")
52
53 # ---- 3. 导出 Markdown ----
54 md_out = output_path / "contract_chapters.md"
55 md_out.write_text(doc.export_to_markdown(), encoding="utf-8")
56 print(f"Markdown 已保存: {md_out}")
57
58 # ---- 4. 导出 HTML (带目录导航) ----
59 toc = "".join(
60     f'<li style="padding-left:{(s["level"]-1)*16}px">'
61     f'<a href="#sec-{i}">{s["heading"]}</a></li>'
62     for i, s in enumerate(sections)
63 )
64 body = "".join(
65     f'<h{min(s["level"]+1,4)} id="sec-{i}">{s["heading"]}</h{min(s["level"]+1,4)}>'
66     + "".join(f"<p>{p}</p>" for p in s["content"])
67     for i, s in enumerate(sections)
68 )
69 html = f'<!DOCTYPE html>'

```

```

70 <html lang="zh-CN"><head><meta charset="UTF-8"><title>合同文档</title>
71 <style>
72 body{{font-family: 'Microsoft YaHei', sans-serif; margin:0; display: flex}}
73 #toc{{width:240px; background:#f7f7f7; padding:20px; position: sticky; top:0; height:100vh; overflow-y: auto}}
74 #toc ul{{list-style: none; padding:0}}
75 #toc a{{text-decoration: none; color: #444; font-size: 13px; line-height: 2}}
76 #toc a: hover{{color: #1677ff}}
77 #content{{flex: 1; padding: 40px 60px; max-width: 860px}}
78 h2, h3, h4{{color: #111; border-bottom: 1px solid #e8e8e8; padding-bottom: 6px}}
79 p{{line-height: 1.9; color: #444}}
80 </style></head>
81 <body>
82 <nav id="toc"><strong>目录</strong><ul>{toc}</ul></nav>
83 <main id="content">{body}</main>
84 </body></html>""
85     html_out = output_path / "contract_visual.html"
86     html_out.write_text(html, encoding="utf-8")
87     print(f"HTML 可视化页面已保存: {html_out}")
88     print(f"\n共解析 {len(sections)} 个章节")
89     return sections
90
91
92 if __name__ == "__main__":
93     import sys
94     file = sys.argv[1] if len(sys.argv) > 1 else "contract.pdf"
95     parse_contract(file)

```

5.2 运行方式

```
</> Bash |
1 # 安装依赖
2 pip install docling
3
4 # 解析 PDF 合同
5 python parse_contract.py contract.pdf
6
7 # 解析 Word 合同
8 python parse_contract.py contract.docx
```

5.3 输出结果说明

1	输出文件	说明
2	contract_chapters.txt	按章节分割的纯文本，章节间有分隔线
3	contract_chapters.md	Markdown 格式，保留标题层级
4	contract_visual.html	带左侧目录导航的 HTML 可视化页面，可直接浏览器打开

六、HybridChunker 按章节分块（适用于 RAG）

如需将合同内容送入向量数据库做语义检索，推荐使用内置的 HybridChunker：

```
</> Python |
```

```
1 from docling.chunking import HybridChunker
2
3 chunker = HybridChunker()
4 for chunk in chunker.chunk(doc):
5     print("所属章节: ", chunk.meta.headings)
6     print("内容: ", chunk.text[:200])
7     print("----")
```

每个 chunk 自带 `meta.headings` （所属标题路径），便于精准溯源。

七、总结与建议

1	维度	评价
2	安装便捷性	★★★★★ pip 一键安装，无外部依赖
3	PDF 解析能力	★★★★★ 基于 AI 模型，识别精度高
4	Word 解析能力	★★★★★ 原生支持 .docx
5	章节结构保留	★★★★★ DoclingDocument 内置层级树
6	表格识别	★★★★★ TableFormer 专用模型
7	可视化输出	★★★★★ 原生 HTML 导出 + 自定义扩展
8	隐私安全	★★★★★ 完全本地运行

推荐方案： 合同文档解析场景优先选用 Docling，使用 PDF/DOCX → DoclingDocument → 按 `section_heading` 标签分章节 → 导出 pdf 可视化页面的完整链路。